

ما هو "التوكن" حقًا؟ خارطة طريق لتوحيد بيانات الذكاء الاصطناعي عبر 12 مزودًا مختلفًا

الإطار المفاهيمي للمقاييس: تصنيف وتوصيف دقيق للبيانات

إن تحقيق هدف البحث المتمثل في بناء متتبع ميزانية شامل وموثوق لنماذج الذكاء الاصطناعي يتطلب تحولاً استراتيجياً جوهرياً في فهم المشكلة. لا يكمن التحدي الأكبر في مجرد "جمع البيانات" من مصادر مختلفة، بل في "توحيد بيانات غير متجانسة دون فقدان معناها". هذا الاستنتاج، الذي تم التوصل إليه بعد تحليل معمق لعشرات حالات دراسة للمزودين، يشكل حجر الزاوية لأي إطار عمل فعال. إن اختلاف المزودين ليس مجرد تفاوت في التفاصيل، بل هو اختلاف في الفلسفة الأساسية لتقديم البيانات، مما يجعل أي محاولة لتطبيق نموذج موحد ثابت ستشهد فشلاً حتمياً. لذلك، فإن الخطوة الأولى والأكثر أهمية هي تطوير إطار مفاهيمي راسخ يمكنه استيعاب هذه الديناميكيات المعقدة. يبدأ هذا الإطار بالقدرة على تحديد المقاييس بدقة وتصنيفها بشكل دقيق، وهو ما يتجاوز مجرد استخدام مصطلح عام مثل "الاستخدام". لقد أظهرت عملية واسعة تتفرع إلى *category* البحث أن كلمة "الاستخدام" نفسها ليست مقياساً واحداً، بل هي ك أنواع رئيسية ذات معاني مختلفة تماماً، وأبرزها: مقاييس الاستخدام، مقاييس معدل الطلب، مقاييس الفوترة، ومقاييس الحصص.

إن هذا التمييز ليس مجرد تمييز أكاديمي، بل هو ضرورة عملية لإنشاء نظام قادر على الفهم يمكن أن *Perplexity*، الصحيح للواقع التشغيلي والمالي. على سبيل المثال، في حالة مزود مثل بينما قد يكون "التكلفة" مدفوعة بوحدة "Query Units" يشير "معدل الطلب" إلى وحدة بدون تصنيف واضح، سيؤدي ذلك إلى انحرافات خطيرة في ⁷⁶ واحدة فقط "successful request". أكد الباحثون أن Anthropic، تقدير التكاليف والقدرة على تحمل الحمل. وبالمثل، في مزود هو الرقم المؤكد الذي يُستخدم في الفوترة، وأن التوكنات التي تظهر في `output_tokens` هذا يعني أن هناك "مقياس". ⁸ الاستجابة قد لا تكون دقيقة بسبب عمليات تحويل داخلية استهلاك "محلي (ما يتم استهلاكه فعلياً) و"مقياس فوترة" مؤكد (ما يتم فرضه). إن فصل هذين هو الذي `billed_units` تم التأكيد على أن Cohere، المقاييس أمر حاسم. وبالمثل، في مزود هذا يؤكد مرة ²¹ يمثل التوكنات المدفوعة فعلياً، وقد يختلف عن عدد التوكنات الخام المُحسَب "أخرى على أن "التوكن المنطقي" ليس بالضرورة "التوكن المدفوع".

لذلك، يجب أن يبدأ أي تصميم نموذج بيانات أو جدول بيانات بمجموعة تصنيف المقاييس. هذا النظام يجب أن يكون جزءاً لا يتجزأ من الهيكل العام، ويحدد لكل قيمة يتم جمعها نوعها الأساسي. يمكن تفصيل هذا التصنيف كما يلي:

- **هذه هي المقاييس التي تقيس كمية العمل (Usage Metrics):** **مقاييس الاستخدام** (عدد الطلبات) `REQUEST_COUNT`، (عدد التوكنات في المدخلات) `INPUT_TOKEN_COUNT`، (عدد التوكنات في المخرجات) `OUTPUT_TOKEN_COUNT`، (عدد الصور) `IMAGE_COUNT`، و (ثواني الصوت) `AUDIO_SECONDS` متخصصة مثل (GPU ثواني تشغيل) `GPU_SECONDS`، و (عدد الأحرف) `CHARACTER_COUNT`. هذه المقاييس هي الأساس الذي يتم بناء التقديرات المالية عليه.
- **هذه المقاييس تقيس سرعة أو تكرار (Rate Metrics):** **مقاييس معدل الطلب**. الاستخدام ضمن فترة زمنية معينة. وهي تحدد سرعة "التشغيل" وليس الكمية الإجمالية. `Tokens Per Minute (TPM)`، و `Requests Per Minute (RPM)` أمثلة شائعة تشمل ومع ذلك، كما أظهر ^{1 2} `Tokens Per Day (TPD)`، و `Requests Per Day (RPD)` يمكن أن يكون هذا المجال أكثر تعقيداً بكثير، حيث يستخدم وحدات مثل Groq، مزود `ASH` (Requests per second in a rolling hour window) و `ASD` (Requests per day in a rolling day window) ⁴⁴. وهذا يفرض على النموذج أن يكون مرناً بما يكفي لاستيعاب "مجموعة واسعة من الوحدة" المرتبطة بكل "معدل".
- **هذه هي المقاييس التي تستخدم مباشرة في (Billing Metrics):** **مقاييس الفوترة** حساب الفاتورة النهائية. غالباً ما تكون هذه المقاييس هي الأكثر أهمية للمحاسبين والمديرين الماليين. قد تكون هذه المقاييس متطابقة مع مقاييس الاستخدام، ولكنها قد بسعر `cached input tokens` يتم حساب OpenAI، تختلف أيضاً. على سبيل المثال، في وفي بعض الحالات، قد تكون الفاتورة مدفوعة ¹⁵ `uncached input tokens` مختلف عن حيث تُحسب Perplexity من Search API بوحدة مختلفة تماماً عن وحدة الاستخدام، كما في لذلك، من الضروري جداً تمييز "مقياس" ⁷⁶ الواحدة "successful request" الفاتورة بناءً على "الاستخدام الفعلي" عن "مقياس الفوترة".
- **هذه المقاييس تمثل الحد الأقصى المسموح به من (Quota Metrics):** **مقاييس الحصص** الاستخدام ضمن نطاق معين. قد تكون الحصص محدودة بعدد الطلبات أو حتى المبلغ المالي (`TOKENS_LIMIT`) أو عدد التوكنات (`REQUESTS_LIMIT`) `model` حدوداً على مستوى AWS Bedrock على سبيل المثال، توفر (`SPEND_LIMIT`) `organization + region + endpoint` ⁴⁵ Anthropic بينما تضع، `model` فهم طبيعة الحصص (هل هي مالية أم تشغيلية؟ هل هي لنموذج معين أم ⁸ `model` `Too Many Requests` لمنظمة بأكملها؟) هو أمر حاسم لمنع حدوث أخطاء ⁴²⁹.

بالإضافة إلى هذا التصنيف، يجب أن يأخذ النموذج في الاعتبار عدة متغيرات أساسية لكل مقياس والتي سيتم تفصيلها في الجداول التقنية لاحقاً. هذه المتغيرات هي ما يمنح النموذج قدرته على كما ذكر الباحث، لا يمكن تخزين (Scope). **النطاق** التعامل مع التنوع. أول هذه المتغيرات هو لمشروع معين؟ أم لمنظمة بأكملها؟ RPM دون تحديد نطاقه. هل هو 5000 RPM رقم "5000" ك ليست دائماً مباشرة؛ quota owner و API Key أم لنموذج معين تحت منظمة؟. العلاقة بين

لكن الحد قد يكون مطبقاً على مستوى Workspace، قد ينتمي إلى مستخدم داخل API Key في billing account و project أو على Anthropic، بأكمله كما في organization لذلك، يجب أن يكون "النطاق" كياناً منفصلاً ومتعدد الأبعاد في النموذج، يتكون من . Gemini Provider, Account, Organization, Project, Workspace, ModelOffering, Model, Endpoint, و Region .

لقد تم تصنيف المصادر إلى عدة أنواع. (Data Source) **مصدر البيانات** المتغير الثاني هو لتحديد مدى موثوقيتها وتوقيتها. المصدر المباشر والأكثر موثوقية هو الحصول على البيانات أما . 70 "A" أثناء التنفيذ، إما من الرأس أو من الجسم. هذا هو المصدر API مباشرة من استجابة إدارية أو حسابية خاصة، والتي غالباً ما تتطلب API فهو الوصول إلى نقاط نهاية ، "B" المصدر وهو البيانات المستخرجة من وثائق رسمية ثابتة أو ، "C" ثم يأتي المصدر . 71 صلاحيات إضافية والتي تكون أقل موثوقية لأنها ، ("R") أو مشتقة محلياً ("L") وهناك مصادر محلية . 65 شبه ثابتة تحديد المصدر بدقة لكل مقياس يسمح للنظام بتقييم "درجة" . 70 تعتمد على الافتراضات المحلية الحادثة و"قابلية الثقة" لكل ملاحظة.

هنا يكمن أحد أكبر الاكتشافات العملية. لقد أظهر (Freshness) **درجة الحادثة** المتغير الثالث هو هذا يعني أنه . 23 وجود تأخير محتمل يصل إلى 10 دقائق في تحديث بيانات الفوترة Gemini مزود لا يمكن الاعتماد على "البيانات الحالية التي يقدمها المزود" لتكون متزامنة تماماً مع "الملاحظة لكل freshness المحلية" التي يقوم بها التطبيق. وبالتالي، أصبح من الضروري إضافة حقل ملاحظة لتحديد مدى قربها من الواقع الزمني الحالي . هذا التأخير الكبير يفسر لماذا يفضل العديد المتأخرة Google Cloud Console في الوقت الفعلي على بيانات OpenAI من المستخدمين بيانات . 23

في بعض الحالات، مثل مزود . **"النطاق غير المعروف"** أخيراً، يجب أن يدرك النظام وجود "قد لا يكون هناك حد رقمي ثابت يمكن قراءته، بل قد يكون "مرونة/أداء مضمون ، Together AI وفي . 72 "token" وليس "GPU-second" قد يكون السعر الأساسي هو ، Fireworks وفي مزود . 54 في هذه . 21 "production key" و "trial key" قد تكون هناك فروقات معقدة بين ، Cohere مزود بدلاً من محاولة تخمين القيمة أو اعتبارها خطأً في UNKNOWN الحالات، يجب على النظام أن يعيد التطبيق . هذا الاعتراف بالحدود هو ما يميز نظاماً ذكياً عن نظام بسيط.

باختصار، بناء إطار عمل فعال لا يبدأ بكتابة كود، بل يبدأ بتطوير هذا الإطار المفاهيمي الغني. إنه يبدأ بالاعتراف بأن البيانات من مزودي الذكاء الاصطناعي ليست متجانسة، وأن التحدي الحقيقي يكمن في فهم هذه الديناميكيات وترجمتها إلى نموذج بيانات مرّن وقوي. إن الانتقال من التفكير في "جدول بيانات واحد" إلى التفكير في "نموذج ملاحظة قائم على الموارد" هو الخطوة التالية، المنطقية، حيث يتمثل كل مقياس كملاحظة غنية بالمعلومات تشمل نوعها، وقيمتها، ووحدتها، ونطاقها، ومصدرها، ودرجة حديثها. هذا هو الأساس الذي سيتم بناء عقد موحد للمزودين عليه، والذي سيوفر القابلية للتوسع والقوة اللازمة لمواجهة تعقيدات السوق المتغيرة باستمرار.

مقاييس الاستخدام والتقييم: معايير قياس الفعل والمبلغ المستحق

تعتبر مقاييس الاستخدام والتقييم العمود الفقري لأي نظام متتبع ميزانية، فهي تجيب على السؤالين الأساسيين: "ما الذي تم فعله؟" و"كم كلف هذا العمل؟". ومع ذلك، وكما تم تسليط الضوء عليه في الإطار المفاهيمي، فإن الإجابة على هذين السؤالين ليست بسيطة ولا موحدة عبر مزودي الخدمات. يتطلب الأمر تفكيك كل مقياس إلى مكوناته الأساسية: التعريف، الوحدة النطاق، المصدر الدقيق، متطلبات الصلاحية، درجة الحداثة، والقدرة على المطابقة المالية. يوضح الجدول التالي تحليلاً مفصلاً لهذه المقاييس الرئيسية بناءً على الأدلة المتوفرة، مما يخلق أساساً تقنياً واضحاً يمكن البناء عليه.

المقياس	التعريف	الوحدة	النطاق (Scope)	مصدر البيانات الدقيق (Endpoint/ Header/Body)	متطلبات الصلاحيات	درجة الحداثة	إمكانية المطابقة المالية
Input Token Count	عدد التوكنات في جزء المدخلات (prompt) لكل طلب.	tokens	Provider, Organization, Model, Endpoint	Header: OpenAI (x-content-token-count-input), Anthropic (anthropic-input-tokens), Mistral (X-Content-Token-Count-Input) Body: OpenAI (prompt_tokens), Anthropic (input_tokens), Mistral (prompt_tokens)	API Key	عالية (من الرأس) / متوسطة (من الجسم)	نعم، من خلال مقارنتها مع بيانات CUR من AWS ⁵⁸ Usage API من OpenAI/Gemini ³⁴ .
Output Token Count	عدد التوكنات في جزء المخرجات (completion) لكل طلب.	tokens	Provider, Organization, Model, Endpoint	Header: Anthropic (anthropic-output-tokens) Body: OpenAI (completion_tokens), Anthropic (output_tokens), Mistral (completion_tokens), Gemini (candidate/output tokens)	API Key	عالية (من الرأس) / متوسطة (من الجسم)	نعم، يمكن مطابقتها مع بيانات الفوترة. في Anthropic، يعتبر output_tokens هو الرقم المؤكد ⁸ للمدفوعات.
Total Token Count	مجموع عدد التوكنات المدخلة والمخرجة لكل طلب.	tokens	Provider, Organization, Model, Endpoint	Header: - Body: OpenAI (total_tokens), Mistral (total_tokens), Groq (total_tokens)	API Key	متوسطة (من الجسم)	نعم، يمكن حسابها محلياً أو استخدامها كقيمة مرجعية للمقارنة.
Cached Input Token Count	عدد التوكنات التي تم استردادها من ذاكرة التخزين المؤقت.	tokens	Provider, Organization, Model	Header: - Body: OpenAI (cached_input_tokens), Anthropic (cache_read_input_tokens), Mistral (cached_tokens)	API Key	عالية (من الجسم)	نعم، تعتبر هذه بيانات مهمة للحساب الدقيق للتوفير عند استخدام التخزين المؤقت.
Cache Write Token Count	عدد التوكنات الجديدة التي تم كتابتها في ذاكرة التخزين المؤقت.	tokens	Provider, Organization, Model	Header: - Body: OpenAI (cache_write_tokens), Anthropic (cache_creation_input_tokens)	API Key	عالية (من الجسم)	نعم، ضرورية للمحاسبة الدقيقة على خدمات التخزين المؤقت حيث تكون لها ²⁶ أسعار منفصلة.
Actual Cost	المبلغ المالي الفعلي المدفوع مقابل الاستخدام خلال فترة زمنية محددة.	USD, EUR, إلخ	Account, Organization, Project	API: OpenAI (/v1/organization/costs), Anthropic (/v1/organizations/cost_report), AWS Bedrock (Cost and Usage Report - CUR), Gemini (Google Cloud Billing Reports) Export: OpenAI (Dashboard Export CSV)	Admin API Key	منخفضة (تأخير) (بأيام)	نعم، هذا هو الغرض الأساسي من المقياس. يتم مقارنة التكلفة الفعلية مع التكلفة المحسوبة من لضمان API طلبات ³¹ دقة الفوترة ³⁷ .
Billable Request Count	عدد الطلبات التي تم احتساب رسوم مقابلها.	requests	Provider, Model, Endpoint	Header: Perplexity (request-unit-incremented) Body: Perplexity (usage.cost.total_cost)	API Key	عالية (من الرأس) / (الجسم)	نعم، يمكن استخدامه كأساس لحساب التكلفة إذا كانت هناك علاقة مباشرة بين عدد الطلبات والتكلفة.
		seconds			API Key	عالية (من الجسم)	نعم، يمكن حساب التكلفة بناءً على

المقياس	التعريف	الوحدة	النطاق (Scope)	مصدر البيانات الدقيق (Endpoint/ Header/Body)	متطلبات الصلاحيات	درجة الحداثة	إمكانية المطابقة المالية
Audio Second Count	مدة الصوت المدفوعة بالتواني.		Provider, Model, Endpoint	Header: - Body: GPT-Live API (voice seconds)			سعر التواني المعلن.
Image Count	عدد الصور المولدة أو المعالجة.	images	Provider, Model, Endpoint	Header: - Body: OpenAI (image token count converted to images), Gemini (image token count converted to images)	API Key	عالية (من الجسم)	نعم، يمكن حساب التكلفة بناءً على سعر الصورة أو حسب عدد التوكنات المرتبطة بها.
GPU Second Count	مدة تشغيل GPU المدفوعة بالتواني.	seconds	Provider, Model, Endpoint	Header: Fireworks AI Body: Fireworks AI (Live view of spend)	API Key	عالية (من الرأس/ واجهة برمجة التطبيقات (الحية)	نعم، يمكن حساب التكلفة بناءً على سعر GPU-second المعلن.

تحليل تفصيلي للمقاييس:

هذه هي المقاييس الأساسية لجميع **Input Token Count** و **Output Token Count** كما API، نماذج اللغة. المصدر الأكثر دقة وتوقيتاً هو الحصول عليها مباشرة من رأس الاستجابة. هذا يسمح بنظام تحكم في الوقت الفعلي. ^{41 8 1} Mistral و Anthropic و OpenAI تفعل المصدر البديل هو جسم الاستجابة، والذي يوفر نفس المعلومات ولكنه أقل توقيتاً لأنه يأتي بعد قد تظهر مقاييس متعددة في الاستجابة، مثل Gemini، اكتمال الطلب. في بعض المزودين مثل مما يتطلب دمجها لتكوين صورة كاملة، candidate/output tokens و prompt tokens هو الرقم المؤكد Anthropic في Output Token Count من المهم ملاحظة أن ²² هذا يسلط الضوء على ضرورة فصل ⁸ للمدفوعات، وأن التوكنات الأخرى قد لا تكون دقيقة "المقياس المستخدم" عن "المقياس المدفوع".

هذا مقياس مفيد للمراقبة العامة وتقدير الحمل الإجمالي. يمكن **Total Token Count**، Groq و Mistral و OpenAI الحصول عليه مباشرة من جسم الاستجابة في معظم المزودين مثل ومع ذلك، بالنسبة للمحاسبة الدقيقة، من الأفضل الاعتماد على ⁴⁰ مما يسهل حسابه محلياً مجموع المكونات الفردية (المدخلات، المخرجات، المخزن المؤقت) بدلاً من هذا المقياس المجمع.

هذه المقاييس تمثل **Cache Write Token Count** و **Cached Input Token Count** أحد أعقد مجالات المحاسبة في الذكاء الاصطناعي. إنها لا تقيس الاستخدام فقط، بل تقيس أيضاً في cached input tokens يتم تضمين OpenAI، فعالية استراتيجية التخزين المؤقت. في Anthropic، أما في ¹⁵ التكلفة الإجمالية ولكن بسعر مختلف (10% من التكلفة القياسية) cache read و (مكتوب) cache creation input tokens: فرض فواتير منفصلة مباشرة في جسم cached_tokens يتم تقديم Mistral، في ²⁶ (مقروء) input tokens

هذا يعني أن النظام يجب أن يكون قادراً على تمييز هذه الفئات المختلفة وتتبعها. ⁴¹ الاستجابة كبيرة في التكاليف 低估 بشكل منفصل لحساب التكلفة الدقيقة. إن عدم القيام بذلك سيؤدي إلى خاصة في التطبيقات التي تعتمد بشدة على التخزين المؤقت.

هذا هو المقياس الأكثر أهمية للمحاسبة المالية. المصدر الأكثر موثوقية هو **Actual Cost** /v1/ استخدام نقاط واجهة برمجة التطبيقات الخاصة بالفوترة أو تقارير التكلفة، مثل في organization/costs في OpenAI، و /v1/organizations/cost_report في Anthropic، و Amazon Bedrock Cost and Usage Report (CUR) في AWS ^{31 34 58} تتطلب درجة. ³⁶ (إداري API مفتاح) هذه النقاط واجهة برمجة التطبيقات دائماً صلاحية إدارية عالية الحدثة لهذا المقياس منخفضة للغاية؛ فالبيانات عادة ما تكون متأخرة بيوم أو أكثر. على سبيل ، ²³ يمكن أن تتأخر حتى 24 ساعة Google Cloud Console المثال، أفاد المستخدمون بأن بيانات هذا التأخير هو العامل الحاسم الذي يدفع الحاجة. ³⁸ يمكن أن تتأخر 10-15 دقيقة Groq وبيانات API إلى نظام قادر على مطابقة التكاليف الفعلية مع التكاليف المحسوبة محلياً من طلبات الفردية. هذه العملية، التي تعرف بالمطابقة المالية، هي الطريقة الوحيدة للتحقق من دقة الفوترة والكشف عن أي اختلافات.

هذه: (Audio, Image, GPU) **و وحدات غير تقليدية Billable Request Count** كيف أن Search API يوضح مثال Perplexity، المقاييس تبرز التنوع الهائل في نماذج التسعير. في نظام "Query Units" طلباً واحداً يمكن أن يمثل طلباً واحداً مدفوعاً ولكن خمس وحدات هذا هو المثال النموذجي الذي يثبت أن "معدل الطلب" و"التكلفة" قد يشيران إلى. ⁷⁶ المعدلات هناك نموذج تسعير مختلف تماماً، Fireworks AI، نفس الطلب ولكن بوحدتين مختلفتين. في مزود مما يتطلب نظاماً مرئياً يدعم وحدات، ("per token") إلى جانب النموذج التقليدي ("GPU-second") وبالمثل، يتم تسعير نماذج الصوت والفيديو بناءً على الثانية، بينما يتم تسعير نماذج. ^{72 51} متعددة إن الاعتراف بهذه التنوعات ودمجها في. ^{18 17} الصور بناءً على الصورة أو التوكن المرتبط بها النموذج هو ما يميز نظام متتبع ميزانية حقيقي عن مجرد جدول بيانات بسيط.

باختصار، تحليل مقاييس الاستخدام والتقييم يكشف عن عالم معقد من البيانات التي تتطلب فهماً دقيقاً. إن بناء جدول تقني مفصل، كما طلب البحث، هو خطوة حاسمة نحو بناء نظام قادر على فهم هذه التعقيدات. يتطلب الأمر الانتقال بعيداً عن فكرة "جدول بيانات واحد" واتباع نهج مرن، يعتمد على "نموذج ملاحظة" يمكنه تخزين كل مقياس مع جميع سماته الفريدة (النطاق، المصدر الوحدة، إلخ). هذا النهج لا يضمن الدقة، بل يضمن أيضاً القدرة على التكيف مع نماذج التسعير الجديدة والمبتكرة التي ستستمر في ظهورها في سوق الذكاء الاصطناعي سريع التطور.

مقاييس الحصص ومعدلات الطلب: قياس السعة والسرعة

بينما تركز مقاييس الاستخدام على "ما تم تنفيذه"، تركز مقاييس الحصص ومعدلات الطلب على "ما يمكن تنفيذه وما هو الحد الأقصى للسرعة". هذه المقاييس حيوية لضمان استقرار التطبيقات التي يمكن أن توقف الخدمة، وإدارة 429 Too Many Requests وتجنب حدوث أخطاء. التكاليف عن طريق التنبؤ بالاستخدام. ومع ذلك، فإن التعقيد هنا لا يقل عن مقاييس الاستخدام. تتنوع وحدات القياس، وتختلف نطاقات الحدود بشكل كبير بين المزودين، وحتى آلية تطبيق الحدود (على سبيل المثال، خزانة الرصاص مقابل نافذة متنقلة) يمكن أن تكون مختلفة. إن فهم هذه الفروق الدقيقة أمر بالغ الأهمية لبناء نظام قادر على التحكم في الوصول والتحكم في التكاليف بشكل فعال.

يوضح الجدول التالي تحليلاً مفصلاً لهذه المقاييس الرئيسية، مع التركيز على المصادر الدقيقة والمتطلبات.

المقياس	التعريف	الوحدة	النطاق (Scope)	مصدر البيانات الدقيق (Endpoint/Header/Body)	متطلبات الصلاحيات	درجة الحدانة	إمكانية المطابقة المالية
Requests Per Minute (RPM)	الحد الأقصى لعدد الطلبات في الدقيقة.	requests/minute	Provider, Organization, Project, Model	Header: OpenAI (x-ratelimit-remaining-requests), Anthropic (anthropic-ratelimit-requests-remaining), Mistral (X-RateLimit-Remaining) API: Anthropic (/v1/organizations/rate_limits), Mistral (/v1/admin/rate-limit)	API Key (للرأس) / Admin API Key (لواجهة البرمجة للتطبيق)	عالية (للرؤوس) / متوسطة (لواجهة البرمجة للتطبيق)	لا تطابق مالية. يُستخدم للتحكم في التنفيذ.
Tokens Per Minute (TPM)	الحد الأقصى لعدد التوكنات المدخلة والمخرجة في الدقيقة.	tokens/minute	Provider, Organization, Project, Model	Header: OpenAI (x-ratelimit-remaining-tokens), Anthropic (anthropic-ratelimit-tokens-remaining) API: Anthropic (/v1/organizations/rate_limits), Mistral (/v1/admin/rate-limit)	API Key (للرؤوس) / Admin API Key (لواجهة البرمجة للتطبيق)	عالية (للرؤوس) / متوسطة (لواجهة البرمجة للتطبيق)	لا تطابق مالية. يُستخدم للتحكم في التنفيذ.
Requests Per Day (RPD)	الحد الأقصى لعدد الطلبات في اليوم.	requests/day	Provider, Organization, Project, Model	Header: OpenAI (x-ratelimit-reset-requests) Documentation: OpenAI (reset at midnight UTC)	API Key	عالية (من الرأس)	لا تطابق مالية. يُستخدم للتحكم في التنفيذ.
Tokens Per Day (TPD)	الحد الأقصى لعدد التوكنات في اليوم.	tokens/day	Provider, Organization, Project, Model	Header: OpenAI (x-ratelimit-reset-tokens) Documentation: OpenAI (reset at midnight UTC)	API Key	عالية (من الرأس)	لا تطابق مالية. يُستخدم للتحكم في التنفيذ.
Spend Limit	الحد الأقصى للمبلغ المالي الذي يمكن إنفاقه في فترة زمنية (عادةً شهر).	USD, EUR, إلخ	Account, Organization, Project	UI/API: OpenAI (in Dashboard), Groq (in UI), AWS Bedrock (via CUR/tags) Alerts: Groq (email alerts)	Admin API Key / Owner Permissions	منخفضة (تعتمد على التحديثات اليدوية أو الإشعارات)	نعم، يمكن التحقق من التكلفة الفعلية مقابل الحد المالي.
Hard Spend Limit	حد مالي صارم يؤدي إلى رفض الطلبات عند تجاوزه.	USD, EUR, إلخ	Organization	Response: OpenAI (429 error with insufficient_quota code) Response: Groq (400 error with blocked_api_access code)	N/A	عالية (في وقت الرد)	نعم، يمنع تجاوز الحد المالي.
Free Tier Entitlement	مقدار الاستخدام المجاني	tokens, credits,	Account, Plan	Documentation: Gemini (free tier limits), Cohere (trial key limits)	API Key	منخفضة (تعتمد على)	نعم، يمكن مقارنة الاستخدام

المقياس	التعريف	الوحدة	النطاق (Scope)	مصدر البيانات الدقيق (Endpoint/Header/Body)	متطلبات الصلاحيات	درجة الحدانة	إمكانية المطابقة المالية
	المقدم من قبل المزود.	requests/day		API: OpenRouter (/api/v1/key for free model requests)		تحديث (الوثائق)	الفعلي مع الحد المجاني.
Credit Balance	الرصيد المتبقي من الرصيد المسبق الشراء.	credits	Account, Organization	API: OpenRouter (/api/v1/credits), Hugging Face (in UI) UI: Together AI (balance cut-off)	Management Key / API Key	منخفضة (تعتمد على التحديثات اليدوية أو إشعارات التوقف)	نعم، يُستخدم للتحقق من الرصيد.

تحليل تفصيلي للمقاييس:

هذه هي المقاييس: **Requests Per Minute (RPM)** و **Tokens Per Minute (TPM)** الأكثر شيوعاً لمعدلات الطلب. المصدر المباشر والأكثر أهمية هو الحصول على القيم المتبقية من توفر هذه المعلومات مباشرة في Mistral و Anthropic و OpenAI. رؤوس الاستجابة على سبيل المثال، رأس . 49 8 1 الرؤوس، مما يسمح للتطبيق بتخفيف الطلبات بشكل استباقي يخبر التطبيق بعدد الطلبات المتبقية OpenAI من x-ratelimit-remaining-requests (rolling window) "ومع ذلك، من المهم ملاحظة أن هذه النافذة قد تكون "متحركة". 4 في النافذة الحالية وليس لها إعادة تعيين ثابتة، مما يعني أن الحساب المحلي يجب أن يكون قادراً على (rolling window) Points of Contact APIs أما المصدر البديل فهو . 2 التعامل مع هذا النوع من الخزنة الرصاصية GET /v1/organizations/rate_limits في Anthropic و GET /v1/admin/rate-limit في Mistral 41 9 هذه الطرق توفر رؤية أعمق حول كيفية تكوين . 30 تتطلب صلاحيات إدارية أعلى الحدود (على سبيل المثال، هل هي مجمعة لعائلة نماذج معينة أم منفصلة لكل نموذج؟) ولكنها .

RPM و هذه المقاييس تشبه: **Requests Per Day (RPD)** و **Tokens Per Day (TPD)** يتم تعيين إعادة تعيين هذه، OpenAI ولكنها تعمل على نافذة زمنية أطول (يوم واحد). في TPM يمكن الحصول على وقت إعادة . 4 المقاييس عادة عند منتصف الليل بالتوقيت العالمي المنسق هذا مهم للتطبيقات التي . 1 x-ratelimit-reset-requests التعيين من الرؤوس مثل يتم توفير، Groq، تحتاج إلى إدارة الحصص على أساس يومي أو أسبوعي. في بعض الحالات، مثل x-ratelimit-limit-requests في رؤوس متعددة مثل reset و remaining و limit قيم x-ratelimit-reset-requests و x-ratelimit-remaining-requests و x-ratelimit-reset-tokens 50.

هذه المقاييس تقع على خط الفصل بين التحكم: **Spend Limit** و **Hard Spend Limit** تكوين حدود مالية Together AI و Groq و OpenAI التشغيلي والتحكم المالي. يمكن للمزودين مثل قد OpenAI عندما يتجاوز المستخدم هذا الحد، يمكن أن يحدث سلوك مختلف. في . 54 38 1 يمنع أي طلبات 429 يؤدي ذلك إلى إرسال تنبيه، أو في حالته "الصارمة"، فإنه يتسبب في رد

هذا هو الميزة الأمنية .³⁸ يتم إيقاف جميع الوصول بمجرد تجاوز الحد ، Groq في .¹ إضافية الأقوى ضد الحوادث غير المتوقعة أو الاختراقات التي قد تؤدي إلى فواتير هائلة. المصدر للتحقق من هذه الحدود غالبًا ما يكون عبر واجهة برمجة التطبيقات الإدارية أو واجهة المستخدم، وقد على سبيل المثال، أفاد المستخدمون بأن نظام الإنذارات في .³⁹ يكون هناك تأخير في تحديثه لديه تأخير يصل إلى 24 ساعة، مما يجعله غير موثوق لمنع الفواتير الكبيرة في Google Cloud في .³³ الوقت الفعلي

هذه المقاييس تتعلق بإدارة الحسابات **Free Tier Entitlement و Credit Balance** المجانية أو المدفوعة مقدماً. تعريف "مجاني" نفسه ليس موحدًا. يمكن أن يكون نموذجًا مجانيًا Hugging Face PRO أو رصيدًا ماليًا شهريًا مثل الذي تقدمه ، Gemini Free Tier بالكامل مثل OpenRouter Free Tier (50 طلبًا / \$2.00) أو عددًا محددًا من الطلبات اليومية مثل ،⁶⁶ يجب على النظام أن يكون قادرًا على التمييز بين هذه الأنماط المختلفة. وبالمثل، يجب .⁵² (يوم يتم إيقاف الوصول عندما يصل الرصيد إلى ، Together AI إدارة الرصيد المسبق الشراء بدقة. في يتم التحقق من الرصيد من خلال نقطة ، OpenRouter في .⁵⁴ الصفر، مما يعمل كحد صارم إن فهم هذه التفاصيل أمر حاسم لتقديم .⁶⁹ /api/v1/credits واجهة برمجة التطبيقات تجربة مستخدم سليمة وتجنب المفاجآت المالية.

تحديات المعمارية في مقاييس الحصص ومعدلات الطلب

1. RPM و TPM قد لا تكون المقاييس محدودة بـ ، Groq كما أظهر مزود :تنوع الوحدات . ASD (Requests per second in a rolling hour window) و ASH (Requests per day in a rolling day window) التقييد من التعقيد .⁴⁴ تضيف طبقة أخرى من التعقيد (النموذج يجب أن يكون مرناً ويمكنه التعامل مع مجموعة من "الوحدة" المرتبطة بكل "معدل").
2. AWS Bedrock، كما ذكرنا سابقاً، لا يمكن فصل الحد عن نطاقه. في :تعقيد النطاق ، Anthropic في .⁴⁵ model + region + endpoint الحدود مرتبطة بالنطاق هذا يتطلب نموذجاً قادراً على تمثيل هذه .⁸ أن تكون الحدود تنظيمية أو فضائية فرعية العلاقات المتداخلة بدلاً من مجرد تخزين رقم في سلسلة نصية.
3. لديهم حدود ديناميكية تزيد مع زيادة Together AI بعض المزودين مثل :آلية التطبيق . هذا يجعل تصنيف الحدود "غير .⁵⁴ الاستخدام المستمر، ولا تأتي في صورة "أقسام" ثابتة معروف" أو "مرنة" نتيجة صحيحة بدلاً من كونها خطأ في التطبيق .
4. "لا يمكننا بعد كل هذا البحث أن نستنتج "الرصيد العالمي المتاح :غياب الحدود العالمية لجميع المزودين. هذا ليس عيباً في النظام، بل هو حد حقيقي للرؤية التي يوفرها المزود بدلاً من تخمين القيمة UNKNOWN نفسه . يجب أن يمثل النظام هذا الغياب بالمفاضلة

في الختام، تتطلب مقاييس الحصص ومعدلات الطلب نهجاً دقيقاً ومتعدد الأوجه. إن بناء جدول تقني مفصل لهذه المقاييس، مع تحديد المصادر الدقيقة والمتطلبات والحدود، هو خطوة ضرورية يجب أن يتجاوز هذا الجدول مجرد قائمة من الميزات ليصبح خريطة طريق للمعاملة مع التعقيدات التشغيلية والمالية التي يفرضها مزودو الذكاء الاصطناعي. إن الاعتراف بالتنوع في الوحدات وفهم تعقيدات النطاق، وتصميم نظام قادر على التعامل مع الحدود الديناميكية وغير المعروفة هو ما يمكن النظام من تحقيق أهدافه الحقيقية: توفير رؤية شاملة وموثوقة للتحكم في الوصول والتحكم في التكاليف.

مقاييس الحسابات والرصيد: إدارة الرصيد والتكاليف

تتطلب إدارة ميزانية نماذج الذكاء الاصطناعي مراقبة دقيقة للحسابات والرصيدات، وهي المقاييس التي تجيب على سؤال "كم من المال لدينا وماذا أنفقنا بالفعل؟". على عكس مقاييس الاستخدام التي تراقب الأحداث الفردية، تركز هذه المقاييس على الوضع المالي الكلي والبيانات التاريخية. إنها تلعب دوراً حيوياً في التخطيط المالي، والتنبؤ بالتكاليف، والتحقق من صحة الفواتير النهائية. ومع ذلك، فإن الحصول على هذه البيانات يمكن أن يكون تحدياً كبيراً، حيث تتطلب الكثير من المصادر صلاحيات إدارية عالية وتتميز بتأخير زمني كبير. إن فهم هذه التحديات وتحديد المصادر الأكثر موثوقية هو أمر بالغ الأهمية لبناء نظام متتبع ميزانية قوي.

يوضح الجدول التالي تحليلاً مفصلاً لهذه المقاييس الرئيسية، مع التركيز على المصادر الدقيقة والمتطلبات والحدود الزمنية.

المقياس	التعريف	الوحدة	النطاق (Scope)	مصدر البيانات الدقيق (Endpoint/Header/Body)	متطلبات الصلاحيات	درجة الحدثة	إمكانية المطابقة المالية
Account Credit Balance	الرصيد المتبقي من الرصيد المسبق الشراء في حساب الميزانية.	USD, EUR, إلخ	Account, Organization	API: OpenRouter (/api/v1/credits), Hugging Face (in UI) UI: Together AI (balance displayed)	Management Key / API Key	منخفضة (تأخر) (تلقائي)	نعم، هذا هو الرصيد الفعلي.
Monthly Spend Limit	الحد الأقصى للإنفاق المسموح به شهرياً.	USD, EUR, إلخ	Organization, Workspace	UI/API: Mistral (Admin Panel), AWS Bedrock (CUR tags) API: Mistral (/v1/admin/spend-limit)	Admin API Key	منخفضة (تعتمد على) (التحديثات اليدوية)	نعم، يمكن التحقق من التكلفة الفعلية مقابل الحد.
Current Monthly Usage	إجمالي الإنفاق الفعلي حتى تاريخه في الشهر الحالي.	USD, EUR, إلخ	Organization, Workspace	API: Mistral (/v1/admin/usage), Together AI (beta usage API) UI: Mistral (Usage dashboard)	Admin API Key	منخفضة (تأخر) (تلقائي)	نعم، يُستخدم للتحقق من التكلفة الفعلية.
Free Monthly Credits	مبلغ الرصيد المجاني الممنوح شهرياً.	USD, EUR, إلخ	Account, Plan	Documentation: Hugging Face (monthly credits), Cohere (trial credits) API: OpenAI (/v1/dashboard/billing/credit_grants)	API Key	منخفضة (تعتمد على) (تحديث) (الوثائق)	نعم، يمكن مقارنة الاستخدام الفعلي مع هذا المبلغ.
Invoice History	سجل الفواتير السابقة.	invoices	Account, Organization	UI: OpenAI (Usage Dashboard -> Invoices), AWS Bedrock (Billing Console)	Admin Access	منخفضة (بيانات) (تاريخية)	نعم، يُستخدم للتحقق من الفواتير السابقة.
Cost Allocation Tags	وسيلة لربط التكاليف بالفرق أو المشاريع.	tags	Resource, Line Item	UI/API: AWS Bedrock (Application Inference Profiles)	IAM Permissions	منخفضة (تأخر) (تلقائي)	نعم، هذا هو الغرض الأساسي من التجزئة.
Usage Report	تقرير مفصل يفصل الاستخدام حسب النموذج والمنتج.	report	Organization, Workspace	API: OpenAI (/v1/usage), Mistral (/v1/admin/usage) UI: OpenAI (Usage Dashboard -> Export CSV)	Admin API Key	منخفضة (تأخر) (تلقائي)	نعم، هذا هو المصدر الرئيسي للمقارنة.
Cost Report	تقرير مفصل يفصل التكاليف حسب النموذج والمنتج والوقت.	report	Organization, Workspace	API: OpenAI (/v1/organization/costs), Anthropic (/v1/organizations/cost_report) UI: AWS Bedrock (Billing Console)	Admin API Key	منخفضة (تأخر) (تلقائي)	نعم، هذا هو المصدر الأكثر موثوقية للمقارنة.

تحليل تفصيلي للمقاييس:

"هذه المقاييس تمثل النقاد: **Free Monthly Credits** و **Account Credit Balance** يمكن للمستخدمين رؤية رصيد الرصيد المسبق الشراء الخاص بهم، Hugging Face المتاحة. في يمكن الحصول على هذا الرصيد من خلال OpenRouter، في 65. مباشرة في واجهة المستخدم أما بالنسبة 69. والتي تتطلب مفتاح إدارة، `/api/v1/credits` نقطة واجهة برمجة التطبيقات يحصلون على \$2.00 شهرياً Hugging Face PRO Users. للرصيد المجاني، فهو يختلف بشكل كبير `/v1/dashboard/` يمنح المستخدمين مخصصات مجانية من خلال OpenAI بينما 66، إن فهم كيفية حساب واستهلاك كل من هذه الرصيد المسبق 35. `billing/credit_grants` الشراء والرصيد المجاني أمر حاسم لإدارة الميزانية بشكل فعال.

هذه المقاييس هي أدوات التحكم **Current Monthly Usage** و **Monthly Spend Limit** على سبيل المثال، يسمح للمدراء بتعيين حد إنفاق شهري على مستوى Mistral. المالي الرئيسية يمكن الحصول على هذا الحد من خلال واجهة 44 43 المنظمة وعلى مستوى المساحة العاملة 42. `/v1/admin/spend-limit` المستخدم أو من خلال واجهة برمجة التطبيقات الإدارية `/v1/` وبالمثل، يمكن الحصول على استهلاك التكلفة الفعلي باستخدام واجهة برمجة التطبيقات يتم تحقيق هذا التحكم بشكل AWS Bedrock، في 41. والتي تقدم تفصيلاً دقيقاً `admin/usage`، `Profiles of Application` مختلف باستخدام تجزئة التكاليف. عن طريق تطبيق تجزئة على AWS Cost يمكن للمديرين ربط التكاليف بالفرق أو المشاريع المختلفة وإظهارها في "Inference"، هذا يتيح رؤية مفصلة دون الحاجة إلى حدود إنفاق صارمة، ولكنه يتطلب تكويناً 62. AWS Explorer 62. إضافياً في

هذه المقاييس هي أساس عملية المطابقة المالية. إنها **Cost Report** و **Usage Report** تقدم نقطتي واجهة برمجة التطبيقات الممتازتين OpenAI. تمثل "صورة" موثوقة لما أنفقه المزود توفر Anthropic 34 35. (للتكلفة) `/v1/organization/costs` و (للمستخدم) `/v1/usage` والذي يوفر تفصيلاً دقيقاً لكل بند، `/v1/organizations/cost_report` تقرير التكلفة عبر هو المصدر الوحيد (CUR) يعد تقرير التكاليف والاستخدام، AWS Bedrock، في 31. من البنود `<em>-input-tokens`، و `</em>-output-tokens`، حيث يفصل بين `<em>-cache-read-input-token-count`، و `</em>-cache-write-input-token-count` الفردية مع التكلفة API إن المقارنة بين التكلفة المحسوبة محلياً من طلبات 58. العملية المعلنة في هذه التقارير هي الخطوة الحاسمة للتحقق من دقة الفوترة وتصحيح أي أخطاء.

تحديات المعمارية في مقاييس الحسابات والرصيد

- المصدر الأكثر موثوقية لهذه المقاييس هو واجهة برمجة **متطلبات الصلاحيات العالية**. مثل `/v1/organization/cost_report` في Anthropic أو `/v1/admin/usage` في Mistral، مفتاح إداري API هذا يمثل حاجزاً للأمان، ولكنه يضمن أن هذه 41 30 (m bắt đầu ب sk-ant-admin...) هذا يمثل حاجزاً للأمان، ولكنه يضمن أن هذه

البيانات الحساسة المالية لا يمكن الوصول إليها إلا من قبل الأشخاص المصرح لهم. يجب على النظام تصميم آلية آمنة لتخزين وإدارة هذه المفاتيح الإدارية.

2. هذه هي أكبر تحدي أمام هذه المقاييس. بيانات الفوترة ليست: **التأخير الزمني** (Delay) في AWS Bedrock، يمكن أن تتأخر البيانات حتى 10 دقائق، Google Gemini، في يمكن أن يكون التأخير، Together AI، في 62 يمكن أن يستغرق تفعيل التجزئة 24-48 ساعة هذا التأخير يعني أن هذه المقاييس غير مناسبة للتحكم في التنفيذ. 61 حتى ساعة واحدة الفوري، ولكنها ضرورية للتدقيق والمطابقة المالية. يجب على النظام أن يكون واعياً بهذا التأخير وأن لا يعتمد عليها للتحكم في الوصول.
3. لا يوجد مصدر واحد موحد لهذه البيانات. قد يكون المصدر هو نقطة واجهة: **تنوع المصادر**. برمجة التطبيقات الإدارية، أو واجهة المستخدم، أو تقرير تحميله من لوحة التحكم. هذا يعني أن النظام يجب أن يكون قادراً على التفاعل مع مصادر متعددة وغير متجانسة. على سبيل واستدعاء واجهة برمجة، Amazon S3 من CUR المثال، قد يحتاج النظام إلى قراءة بيانات AWS. وعرض بيانات من واجهة المستخدم لـ OpenAI، التطبيقات الإدارية لـ
4. توجد تكاليف، AWS Bedrock الفواتير ليست مجرد تكلفة إجمالية. في: **تعقيد الفواتير**. متعددة: استعلام النموذج الأساسي، وقراءة/كتابة ذاكرة التخزين المؤقت، والبحث إن فهم هذه البنود المختلفة وجمعها. 56 والترتيب، والدرعات، ومحركات الأعمال، والتخزين 58. هو عملية معقدة تتطلب تحليل دقيق CUR بشكل صحيح من

في الختام، تتطلب مقاييس الحسابات والرصيد نهجاً مدروساً يركز على الموثوقية بدلاً من الحداثة. إنها تمثل الجانب المحاسبي من النظام، وتتطلب الوصول إلى بيانات حساسة وغالباً ما تكون متأخرة. إن بناء جدول تقني مفصل لهذه المقاييس، مع تحديد المصادر الدقيقة والمتطلبات والتأخيرات، هو خطوة ضرورية لضمان أن النظام يمكنه إنجاز مهمته الأخيرة: التحقق من صحة الفواتير النهائية وتقديم رؤية مالية نهائية وموثوقة للميزانية. إن الاعتراف بالحوافز الأمنية (مثل متطلبات الصلاحيات العالية) والتحديات العملية (مثل التأخير الزمني) هو ما سيمكن النظام من العمل بشكل موثوق وفعال.

"التحديات المعمارية: نطاق البيانات، تعقيدات "مجاني والنطاق غير المعروف"

بعد تحديد وتصنيف المقاييس الرئيسية وتوثيق مصادرها الدقيقة، يبرز ثلاثة تحديات معمارية رئيسية تتطلب حلولاً مبتكرة ونماذج بيانات مرنة. هذه التحديات ليست مجرد تفاصيل فنية، بل هي جوهر التعقيد الذي يميز هذا المشروع. إنها تتعلق بكيفية تمثيل البيانات بشكل صحيح، وكيفية التعامل مع الحالات الحافة المعقدة، وكيفية إظهار الفجوات في المعرفة بدلاً من محاولة ملؤها.

بالتخمين. إن النجاح في تجاوز هذه التحديات هو ما سيفرق بين نظام متتبع ميزانية جيد ونظام استثنائي قوي وموثوق.

(The Structural Complexity of Scope) التحدي الأول: التعقيد الهيكلي للنطاق

إن أحد أكبر الاكتشافات التي تمخض عنها البحث هو أن "النطاق" ليس مجرد سلسلة نصية دون تحديد "RPM remaining" بسيطة، بل هو كيان هيكلي معقد ومعقد. إن محاولة تخزين رقم نطاقه الكامل هو أمر عديم القيمة. كما أشار الباحث، يجب أن يصبح "النطاق" كياناً مستقلاً في quota owner و API Key النموذج، يمكن أن يتكون من عدة مكونات متداخلة. العلاقة بين ليس بالضرورة هو الكيان الذي يتحمل الحد. على API Key. هي الأكثر وضوحاً في هذا الصدد: سبيل المثال:

- ولكن يمكن ، `organization` الحدود يمكن أن تكون مطابقة على مستوى Anthropic: هذا يعني أن . 8 `workspace` للعملية الافتراضية أن تملك حدوداً خاصة بها على مستوى التي تم توجيه الطلب إليها workspace الحد الفعلي الذي يشعر به المستخدم يعتمد على.
- هذا يعني أن . 13 7 الحدود مرتبطة بالمشروع والحساب الذي تم ربطه بالفاتورة Gemini: الحد يعتمد على سياق الفوترة وليس فقط على المفتاح.
- هذا يعني أن . 45 `model + region + endpoint` الحدود مرتبطة بالنطاق AWS Bedrock: حتى لو كانا يستخدمان نفس ، `bedrock-mantle` و `bedrock-runtime` حركة المرور إلى . 45 النموذج، قد تكون مقيدة بحدود منفصلة.
- يمكن أن يكون المالك المالي هو "Custom Provider Key" في نموذج Hugging Face: نفسها Hugging Face وليس (upstream provider) المزود الأصلي . 65

scope هذا التعقيد يتطلب نموذجاً قادراً على تمثيل هذه العلاقات المتعددة الأبعاد. بدلاً من حقل مع مكونات مثل Scope نصي واحد، يجب أن يكون لدينا كيان:

- `Provider`
- `Account` (الحساب ID)
- `Organization` (المنظمة ID)
- `Project` (المشروع ID)
- `Workspace` (المساحة العاملة ID)
- `Credential` (API مفتاح ID)
- `ModelOffering` (معرف عرض النموذج)
- `Model` (معرف النموذج)
- `Endpoint` (معرف نقطة النهاية)
- `Region` (المنطقة)

تكون دقيقة تماماً من (Observation) "إن هذا النهج، كما اقترح الباحث، يضمن أن كل "ملاحظة فإننا نعرف بالضبط ما إذا كان 5000 Tokens Per Minute حيث السياق. عندما نرى رقم

هذا الرقم مخصصاً لمشروع معين، أو لمنظمة بأكملها، أو لنموذج معين تحت منظمة. هذا المستوى من الدقة هو الذي يسمح بالتحكم الفعال في الوصول والتحكم في التكاليف.

(The Complexity of 'Free') "التحدي الثاني: تعقيدات "مجاني"

كلمة "مجاني" ليست مفردة في عالم الذكاء الاصطناعي. إنها تغطي طيفاً واسعاً من المفاهيم، `is_free` ويجب على النظام أن يكون قادراً على التمييز بينها بدقة. محاولة استخدام ملكية واحدة فقط لتمثيل كل شيء ستؤدي حتماً إلى سوء فهم ومشاكل في المحاسبة. الأنماط حيث، "Gemini Free Tier" مثل: **نموذج مجاني بالكامل**. المختلفة التي تم تحديدها تشمل: 1 في هذه. 6 يتم توفير نماذج معينة مجاناً مع حدود صارمة على معدل الطلب والطلب اليومي **رصيد مالي**. الحالة، يجب على النظام أن يراقب كل من الاستخدام والحدود المجانية. 2 "Cohere" أو، 66 الذين يحصلون على \$2.00 شهرياً "Hugging Face PRO Users" مثل: **شهري** هنا، يجب على النظام تتبع الرصيد. 21 التي تأتي مع حدود شهرية على عدد الطلبات "Trial Keys"، "OpenRouter Free Tier" مثل: **حد يومي لعدد الطلبات**. المتبقي من هذا الرصيد المالي. 3 **نموذج**. هذا يركز على الحد التشغيلي بدلاً من المالي. 4. 52 الذي يوفر 50 طلباً مجانياً في اليوم الذي له حدود مختلفة عن المفتاح الإنتاجي، بما في ذلك حد، "Cohere Trial Key" مثل: **تجريبي** يجب على النظام أن يكون قادراً على تحديد نوع المفتاح. 21 شهري على عدد الطلبات مثل نموذج "مدفوع فقط للمخرجات" حيث تكون: **مجاني لبعض الأجزاء فقط**. المستخدم. 5 المدخلات مجانية ولكن المخرجات مدفوعة. أو نموذج "مجاني حتى استنفاد الرصيد" حيث يكون 13. مجاناً في البداية ثم يتحول إلى نموذج مدفوع

عن معلومات الحد المجاني. يجب على `is_free` لحل هذا التحدي، يجب أن يتم فصل الملكية النموذج أن يحتوي على هيكل بيانات قادراً على تمثيل كل نوع من هذه الأنماط. على سبيل يمكننا أن نرى، `is_free: true` المثال، بدلاً من

```
"entitlement": {
  "type": "monthly_credit",
  "amount": 2.00,
  "currency": "USD",
  "remaining": 1.50
}
```

أو

```
"entitlement": {
  "type": "daily_request_limit",
  "limit": 50,
  "reset_time_utc": "2026-10-26T00:00:00Z"
}
```

"gratis" هذا النهج يوفر مرونة كبيرة ويضمن أن النظام يمكنه التعامل مع أي شكل من أشكال "ال" التي قد يقدمها المزود.

(The Unknown Scope) التحدي الثالث: النطاق غير المعروف

في حين أن جمع البيانات هو هدف البحث، فإن فهم ما لا يمكن جمعه هو بنفس الأهمية. إن الاعتراف بالحدود الحقيقية للمعلومات التي يوفرها المزود هو ما يميز نظاماً ذكياً عن نظام بسيط. "هناك حالات متعددة في البحث أظهرت أن بعض المقاييس قد تكون "غير معروفة" أو "ديناميكية" أو "غير متاحة" على الإطلاق.

- "يوضح أن حدوده قد تكون "مرونة/أداء مضمون Together AI مزود مثل: **حدود ديناميكية** هذا يعني أن النظام يجب أن يتعامل مع الحد كقيمة متغيرة تزداد مع 54 بدلاً من رقم ثابت. زيادة الاستخدام، وليس كقيمة ثابتة يمكن مقارنتها مباشرة.
- في بعض الحالات، قد لا يكون هناك طريقة مباشرة للحصول على **حدود غير متاحة** الرصيد المتاح "العالمي. هذا ليس عيباً في النظام، بل هو حد للرؤية التي يوفرها المزود" بدلاً من محاولة تخمين `UNKNOWN` نفسه. يجب أن يمثل النظام هذا الغياب بالمفاضلة القيمة أو إرجاع خطأ.
- قد يكون السعر الأساسي هو Fireworks AI، كما في مزود: **أساليب تسعير غير تقليدية** هذا يتطلب من النظام أن يكون قادراً على التعامل مع 72 "token" وليس "GPU-second" وحدات تسعير غير مألوفة.

لإدارة هذا "النطاق غير المعروف"، يجب على النظام أن يحتوي على مفاضلات واضحة في واجهة التي تفترض أنها `get_remaining_quota()` برمجة التطبيقات الخاصة به. بدلاً من وظيفة ستعود دائماً برقم، يمكن أن تكون الإرجاعات هي:

- `{"status": "SUCCESS", "value": 5000, "unit": "requests"}`
- `{"status": "UNSUPPORTED", "reason": "No real-time quota available"}`
- `{"status": "ELASTIC", "description": "Limits are dynamic and scale with usage"}`
- `{"status": "UNKNOWN", "reason": "Insufficient permissions or provider limitation"}`

هذا النهج، كما اقترح الباحث، يوفر واجهة متكاملة للمزودين ويسمح للتطبيقات التي تستخدم النظام بالتعامل مع كل حالة بشكل مناسب بدلاً من فرض افتراضات خاطئة.

في الختام، إن التغلب على هذه التحديات المعمارية الثلاثة هو ما سيضمن بناء إطار عمل قوي ومستدام. يتطلب الأمر الانتقال من التفكير في جداول بيانات بسيطة إلى تصميم نموذج بيانات "معقد وغني، قادر على تمثيل العلاقات المتعددة الأبعاد للنطاق، والتفريق بين أنماط "مجاني متنوعة، والاعتراف بالحدود الحقيقية للمعلومات المتاحة. إن الاستثمار في بناء هذا النموذج القوي في البداية سيوفر أعظم فوائد على المدى الطويل من خلال تقليل الأخطاء، وزيادة الدقة، وزيادة قابلية التوسع، مما يسمح للنظام بالتكيف مع التعقيدات المستقبلية في سوق الذكاء الاصطناعي.

الخلاصة الاستراتيجية: بناء عقد موحد للمزودين بناءً على الأدلة الموثقة

بعد التحليل المعمق لمقاييس الاستخدام، ومعدلات الطلب، والحجوزات المالية، والتحديات المعمارية الأساسية، يمكن الآن استخلاص رؤية استراتيجية واضحة لبناء منتج ميزانية نماذج الذكاء الاصطناعي. لقد تحول المشروع من مجرد مهمة تقنية لجمع البيانات إلى مشروع مفاهيمي يتطلب إطار عمل منهجي ومرن. إن النتيجة النهائية لهذا البحث ليست مجرد قائمة بالمزودين الذين يتم دعمهم، بل هي "عقد موحد للمزودين" -- مجموعة من المبادئ والقواعد التي تحدد كيفية تفاعل النظام مع أي مزود جديد بناءً على الأدلة الموثقة التي تم جمعها. هذا العقد هو ما يضمن قابلية التوسع والقوة والدقة.

إن الخطوة الأولى نحو تحقيق هذا العقد هي تبني "نموذج ملاحظة" القائم على الموارد بدلاً من Observation، جداول بيانات خاصة بكل مزود. يجب أن يكون الكائن الأساسي في النظام هو والذي يمثل ملاحظة واحدة لحدث ما. يجب أن يتضمن هذا الكائن جميع المكونات التي تم تحديدها في هذا التقرير:

- `QUOTA` أو `USAGE`، `RATE`، `BILLING`، نوع المقياس، مثل: `Metric`.
- القيمة العددية للملاحظة: `Value`.
- `USD`، `seconds`، `requests`، `tokens` وحدة القياس، مثل: `Unit`، `GPU\_seconds`.
- `Provider` كيان متعدد الأبعاد يحدد سياق الملاحظة بدقة، بما في ذلك: `Region`، `Organization`، `Project`، `Model`، `Endpoint`.
- `Endpoint` أو `Header`، `Body` مسار البيانات الدقيق، سواء كان: `Source`.
- `Authority` (Direct Runtime)، `B` (Admin API)، `A` مستوى الثقة في المصدر، مثل: `C` (Official Docs)، `L` (Local Observation)، أو `R` (Locally Derived) ⁷⁰.
- مقياس لتوقيت الملاحظة، وهو أمر حاسم نظراً للتأخيرات في البيانات: `Freshness` المالية.
- وقت حدوث الملاحظة: `Timestamp`.
- المزود الذي جاء منه الملاحظة: `Provider`.

هذا النهج القائم على الموارد هو ما يمنح النظام المرونة اللازمة لاستيعاب التنوع الهائل في واجهات برمجة التطبيقات ومصادر البيانات.

بناءً على هذا النموذج، يجب تصميم "عقد مزود" صارم وواضح. بدلاً من واجهة برمجة التطبيقات يجب أن يحدد العقد مجموعة من الطرق، `listModels()` و `chat()` العامة البسيطة مثل المحددة التي يجب أن يدعمها كل مزود. يجب أن تعود كل وظيفة بحالة واضحة تشير إلى ما إذا يدعم (SUPPORTED: كانت دعمها ممكناً، وما هي الشروط اللازمة. يمكن أن تكون هذه الحالات (بدعم ولكن يتطلب صلاحية إدارية) REQUIRES_ADMIN، (لا يدعم) UNSUPPORTED، (بوضوح لا يمكن تحديد الدعم بناءً على الأدلة) UNKNOWN أو، (يتطلب مدخلات يدوية) MANUAL_ONLY.

الحالية). على سبيل المثال، طريقة مثل

و OpenAI لـ SUPPORTED قد تعود بـ `get_rate_limits_from_response_headers()` الذي لا يوفر هذه المعلومات في الرؤوس AWS Bedrock لـ UNSUPPORTED ولكن Anthropic، بشكل مباشر.

علاوة على ذلك، يجب أن يصبح "نظام تصنيف المقاييس" () جزءاً لا يتجزأ من هذا العقد. يجب أن Rate أم Usage Metric يكون لدى كل مزود قدرة على تحديد نوع كل مقياس يرسله. هل هو ؟ وما هي وحدته الأساسية؟ هذا التصنيف هو ما يسمح للنظام Billing Metric أم Metric، BILLABLE_TOKEN_COUNT، بتنفيذ المنطق الصحيح. على سبيل المثال، إذا كان المقياس هو TOKENS_REMAINING، فإن النظام يجب أن يضيفه إلى حساب التكلفة. إذا كان المقياس هو "يجب أن ينقصه من الرصيد المتاح. بدون هذا التصنيف، سيكون للنظام "بيانات" ولكن ليس "فهمًا" لها.

() : إن التطبيق العملي لهذا العقد يمكن تلخيصه في التصنيف المقترح للمزودين:

- حيث يمكن Mistral و Anthropic و OpenAI مزودون مثل : **الفئة الأولى (الملاحظ)** الحصول على كمية كبيرة من البيانات الحية والحسابية بدقة عالية من الرؤوس وواجهات برمجة التطبيقات الإدارية. يجب أن يركز الجهود هنا على استخراج هذه البيانات بأفضل طريقة ممكنة.
- حيث Perplexity و Cohere و AWS Bedrock مزودون مثل : **الفئة الثانية (الجزئي)** CUR، تكون البيانات متاحة ولكنها موزعة عبر مصادر متعددة (سجلات استدعاء، تقارير رؤوس، جسم الاستجابة) أو تكون الحدود مرتبطة بالخطط/الواجهات. يجب على النظام أن يكون قادرًا على دمج هذه المصادر المختلفة في نموذج واحد متماسك.
- حيث تكون بعض Fireworks و Together AI مزودون مثل : **الفئة الثالثة (الOpaque)** البيانات (مثل الحدود الديناميكية أو نماذج التسعير غير التقليدية) غير متاحة بشكل مباشر أو `UNKNOWN` يجب على النظام أن يتعامل مع هذا الغياب بطريقة ذكية، وإرجاع `ELASTIC` بدلاً من محاولة تخمين القيمة.

في الختام، يمثل هذا البحث الأساس الذي سيوجه بناء نظام متتبع ميزانية قوي ومستدام. لقد أثبت أن التحدي الحقيقي ليس في "جمع البيانات"، بل في "توحيد المعنى غير المتجانس". إن الحل لا يكمن في بناء جسر مباشر بين جميع المصادر، بل في بناء نظام ذكي قادر على فهم وترجمة وتجميع البيانات من مصادر متعددة ومعقدة، مع إدراك كامل لحدود هذه البيانات. إن الخطوة التالية المنطقية، كما اقترحت، هي توثيق هذه العلاقات المحددة بشكل فائق التفصيل في مما سيخلق الأساس المتين الذي سيتم بناء النظام عليه. إن اتباع "Metric-to-Metric" جدول الإطار المفاهيمي والنهج المعماري الذي تم توضيحه في هذا التقرير سيضمن بناء نظام لا يلي المتطلبات الحالية فحسب، بل يمكنه أيضاً التكيف مع التطورات المستقبلية في هذا المجال السريع التغير.

مرجع

1. Rate limits | OpenAI API <https://developers.openai.com/api/docs/guides/rate-limits>
2. OpenAI Rate Limits: Complete Guide to TPM, RPM & Tier ... <https://inference.net/content/openai-rate-limits-guide/>
3. Managing OpenAI Rate Limits at Scale: A Practical Guide <https://www.getmaxim.ai/articles/managing-openai-rate-limits-at-scale-a-practical-guide/>
4. OpenAI API Rate Limits Explained: RPM, TPM, and Usage ... <https://docs.bswen.com/blog/2026-03-10-openai-api-rate-limits-explained/>
5. Rate limits | Gemini API - Google AI for Developers <https://ai.google.dev/gemini-api/docs/rate-limits>
6. Gemini API Free Quota 2025: Complete Guide to Rate ... <https://www.aifreeapi.com/en/posts/gemini-api-free-quota>
7. Gemini API Rate Limits: Tier Quotas and 429 Fixes <https://fast.io/resources/gemini-api-rate-limit/>
8. Rate limits - Claude Platform Docs <https://platform.claude.com/docs/en/api/rate-limits>
9. Rate Limits API - Claude Platform Docs <https://platform.claude.com/docs/en/manage-claude/rate-limits-api>
10. Google Vertex AI Pricing: Complete Enterprise Guide (2026) <https://www.cloudzero.com/blog/google-vertex-ai-pricing/>
11. Gemini Developer API pricing <https://ai.google.dev/gemini-api/docs/pricing>
12. Google Vertex AI Pricing 2026: Full Cost Breakdown <https://www.opslyft.com/blog/google-vertex-ai-pricing>
13. Google Gemini API and Vertex AI licensing <https://wiki.licenseware.io/wiki/google-gemini-api-and-vertex-ai-licensing/>
14. Pricing | OpenAI API <https://developers.openai.com/api/docs/pricing>
15. OpenAI API pricing in 2026: every model from Astra to Luna <https://www.cloudzero.com/blog/openai-pricing/>
16. OpenAI API Pricing: Models & Token Costs, How to Save <https://mem0.ai/library/llms-and-models/openai-api-pricing>
17. Help understand token usage with vision API <https://community.openai.com/t/help-understand-token-usage-with-vision-api/893022>

18. Cost optimization | OpenAI API <https://developers.openai.com/api/docs/guides/voice-latency-cost>
19. Plans & pricing | Claude by Anthropic <https://claude.com/pricing>
20. Claude Pricing (2026): Plans, API Rates & Usage Billing <https://kingy.ai/news/usage-based-billing-no-flat-rate-why-anthropics-2026-pricing-shift-changes-everything-for-claude-users/>
21. Claude API Pricing 2026: Opus 4.8, Sonnet 4.6, Haiku 4.5 ... <https://www.metactto.com/blogs/anthropic-api-pricing-a-full-breakdown-of-costs-and-integration>
22. Understand and count tokens | Gemini API <https://ai.google.dev/gemini-api/docs/tokens>
23. Where exactly can I quickly check usage & cost for API? <https://discuss.ai.google.dev/t/where-exactly-can-i-quickly-check-usage-cost-for-api/3558>
24. Pricing - Claude Platform Docs <https://docs.anthropic.com/en/docs/about-claude/pricing>
25. Rate limits - Claude Platform Docs <https://docs.anthropic.com/en/api/rate-limits>
26. Prompt caching - Claude Platform Docs <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>
27. Token counting - Claude Platform Docs <https://docs.anthropic.com/en/docs/build-with-claude/token-counting>
28. Tool use with Claude - Claude Platform Docs <https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview>
29. Claude Platform release notes <https://docs.anthropic.com/en/release-notes/overview>
30. Admin API - Claude Platform Docs <https://docs.anthropic.com/en/api/administration-api>
31. Get Cost Report - Claude API Reference <https://docs.anthropic.com/en/api/admin-api/usage-cost/get-cost-report>
32. Using Gemini API keys - Google AI for Developers <https://ai.google.dev/gemini-api/docs/api-key>
33. How to limit my budget cost say \$30 a month spent for Gemini ... <https://discuss.ai.google.dev/t/how-to-limit-my-budget-cost-say-30-a-month-spent-for-gemini-api/34512>
34. Introducing the Usage API—track API usage and costs ... <https://community.openai.com/t/introducing-the-usage-api-track-api-usage-and-costs-programmatically/1043058>
35. OpenAI Billing and Usage Dashboard: How to Track Spend ... <https://www.toriihq.com/articles/how-to-monitor-spending-openai>

36. OpenAI API Usage: group_by & CSV - AICostPlanner <https://aicostplanner.com/openai-api-usage/>
37. Monitor the usage of Gemini API on Vertex AI <https://discuss.google.dev/t/monitor-the-usage-of-gemini-api-on-vertex-ai/171653>
38. Spend Limits - GroqDocs <https://console.groq.com/docs/spend-limits>
39. Billing FAQs - GroqDocs <https://console.groq.com/docs/billing-faqs>
40. API Reference - GroqDocs <https://console.groq.com/docs/api-reference>
41. Beta Admin Billing Endpoints - Mistral AI Documentation <https://docs.mistral.ai/api/endpoint/beta/admin/billing>
42. Usage metrics with the Admin API - Mistral AI Documentation <https://docs.mistral.ai/admin/admin-api/usage-metrics>
43. Usage and limits | Mistral Docs <https://docs.mistral.ai/admin/billing-usage/usage-limits>
44. Workspace usage and limits | Mistral Docs <https://docs.mistral.ai/admin/workspaces/usage-limits>
45. Quotas for Amazon Bedrock <https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html>
46. Amazon Bedrock endpoints and quotas <https://docs.aws.amazon.com/general/latest/gr/bedrock.html>
47. Mistral Medium 3.5 API Rate Limits & Pricing (July 2026) <https://www.rapidevelopers.com/ai-api-limits-performance-matrix/mistral-medium>
48. AI API Rate Limits Compared: Every Major Provider in 2026 <https://crazyrouter.com/en/blog/ai-api-rate-limits-every-provider-compared-2026>
49. Known limitations | Mistral Docs <https://docs.mistral.ai/resources/known-limitations>
50. Groq API Free Tier Limits in 2026: What You Actually Get <https://www.grizzlypeaksoftware.com/articles/p/groq-api-free-tier-limits-in-2026-what-you-actually-get-uwysd6mb>
51. Groq Pricing In 2026: Every Model, Tier, And Cost Compared <https://www.cloudzero.com/blog/groq-pricing/>
52. OpenRouter Pricing 2026: Plans, Costs, and Hidden Fees <https://www.truefoundry.com/blog/openrouter-pricing>
53. OpenRouter Pricing: Token Costs, Credit Fees and BYOK <https://ofox.ai/blog/openrouter-pricing-hidden-markup-breakdown-2026/>
54. Together AI | Pricing Index <https://flexprice.io/pricing-index/together-ai>
55. Overview - Together AI docs <https://docs.together.ai/docs/serverless/overview>

56. Amazon Bedrock Pricing Explained <https://caylent.com/blog/amazon-bedrock-pricing-explained>
57. Amazon Bedrock pricing in 2026: every model and hidden ... <https://www.cloudzero.com/blog/amazon-bedrock-pricing/>
58. Understanding your Amazon Bedrock Cost and Usage ... <https://docs.aws.amazon.com/bedrock/latest/userguide/cost-mgmt-understanding-cur-data.html>
59. AWS Bedrock Pricing 2026: Models, Throughput, EDP <https://redresscompliance.com/aws-bedrock-pricing-guide-2026>
60. Usage limits & analytics <https://docs.together.ai/docs/billing-usage-limits>
61. Spend controls - Together AI docs <https://docs.together.ai/docs/billing-spend-controls>
62. Track generative AI costs with Amazon Bedrock inference ... <https://aws.amazon.com/blogs/architecture/track-generative-ai-costs-with-amazon-bedrock-inference-profiles/>
63. Build a proactive AI cost management system for ... <https://aws.amazon.com/blogs/machine-learning/build-a-proactive-ai-cost-management-system-for-amazon-bedrock-part-1/>
64. Understanding AWS Bedrock: Basics, Pricing Calculator ... <https://www.finout.io/blog/aws-bedrock-pricing-optimization-guide>
65. Pricing and Billing <https://huggingface.co/docs/inference-providers/en/pricing>
66. Hugging Face Inference - API <https://metronome.com/pricing-index/hugging-face-inference-api>
67. OpenRouter API Reference - Complete Documentation https://openrouter.ai/docs/api_reference/overview
68. API Credit & Rate Limits - Handle 402 and 429 Errors https://openrouter.ai/docs/api_reference/limits
69. Get remaining credits - OpenRouter | Documentation <https://openrouter.ai/docs/api-reference/credits/get-remaining-credits>
70. API keys - Amazon Bedrock - AWS Documentation <https://docs.aws.amazon.com/bedrock/latest/userguide/api-keys.html>
71. Fireworks AI Python API Docs <https://dlthub.com/context/source/fireworks-ai>
72. Fireworks AI Cost Management | Cloud Intelligence™ <https://www.doit.com/integrations/fireworks>
73. Responses API <https://docs.fireworks.ai/guides/response-api>
74. Fireworks AI Cost Monitoring Software & Alerts <https://www.stackspend.app/fireworks-cost-monitoring>
75. Exporting Usage Costs <https://docs.fireworks.ai/accounts/exporting-usage-costs>

76. Perplexity Pricing in 2026 for Individuals, Orgs & Developers <https://www.finout.io/blog/perplexity-pricing-in-2026>